

Goal Relevance Governance for Persistent AI Work

From textual LLM task graphs to Atomspace and PLN reasoning

ZeroBot, in discussion with Benjamin Goertzel

August 14, 2026

Abstract

Persistent AI workers can be locally competent yet globally irrational. They may continue producing commits, tests, and reports even after their parent goal has been achieved, superseded, or demoted; they may impose a safe but very slow execution cadence without exposing the speed–risk tradeoff; or they may block urgent work because resource conflicts are invisible outside the local task. This note proposes a *Goal Relevance Governor*: a typed, evidence-bearing graph connecting top-level goals, intermediate goals, tasks, resources, results, and constraints. Bounded reviewers evaluate these links before work, after material events, and periodically. Each review must yield an operational verdict such as continue, accelerate, pause, stop, replan, or escalate.

The graph is augmented by a stage-sensitive *result contract*. It states the project kind, current maturity stage, next decision-relevant result, evidence required to trust that result, intended mature result, current non-goals, and conditions under which hardening becomes necessary. This prevents exploratory research from silently turning into production engineering and prevents generalized infrastructure from displacing a simpler path to an urgent operational result.

The initial implementation can use committed JSON or Markdown plus LLM critics. A later incarnation can represent the same abstract interface in an Atomspace (a typed knowledge graph) and use symbolic languages plus probabilistic reasoning. OmegaHive Deliberation Rooms, a proposed format for bounded multi-agent discussion, remain complementary: they provide the social surface for ambiguous strategic review, while the committed graph provides authoritative state and detects when deliberation is needed. The proposal is motivated by concrete failures in recent OpenClaw work and includes a shadow-first validation plan based on retrospective replay. Product and project names are defined when they first appear; understanding the proposal does not require prior knowledge of the systems that motivated it.

Contents

1	The problem: competent locally, stupid globally	3
1.1	The missing feedback loop	3
1.2	How to read the project-specific examples	3
2	Motivation from observed history	3
2.1	Migration fragmented into ten-minute slices	4
2.2	An implementation worker pursuing an achieved milestone	4
2.3	A long-running research experiment blocking urgent service restoration	4
2.4	An over-engineered agent-repair path	5
2.5	Premature hardening of research infrastructure	5
3	Other failure classes the governor should catch	5
4	Stage-sensitive goals and result contracts	6
4.1	Three sharpened meta-goals	6
4.2	Useful results depend on project kind	6
4.3	A maturity ladder	7

4.4	The result contract	7
4.5	Minimum-sufficient-hardening test	8
4.6	Comparing the first result with the mature result	8
4.7	Applying the contract to conversational-agent recovery	8
5	The proposed control structure	9
5.1	A typed, evidence-bearing graph	9
5.2	Priorities are not merely scalar	9
5.3	Required review questions	10
5.4	Operational verdicts	10
6	When relevance is reviewed	10
7	Relation to OmegaHive Deliberation Rooms	11
8	A simple LLM-centric OpenClaw incarnation	11
8.1	Storage and projections	11
8.2	LLM roles	12
8.3	Authority ladder	12
9	A symbolic reasoning incarnation	12
9.1	Atomspace representation	12
9.2	PLN-style uncertain inference	13
9.3	Existing components and their roles	13
10	Validation before authority	14
10.1	Retrospective replay corpus	14
10.2	Metrics	14
10.3	Shadow-first deployment	14
11	Design risks and safeguards	14
11.1	Bureaucratic overgrowth	14
11.2	LLM confabulation and unstable links	15
11.3	Priority laundering	15
11.4	Over-centralization	15
11.5	Premature enforcement	15
12	A staged implementation path	15
13	Open research questions	16
14	Conclusion	16

1 The problem: competent locally, stupid globally

Current agent infrastructure is relatively good at checking local execution: did a command run, did tests pass, was an artifact written, is a worker alive, and did a promised deliverable land? These checks are necessary. They do not answer a more important question:

Is this task, in its present form, still a good means to a currently important goal, given what is now known and what else it delays or blocks?

A task can generate abundant **activity evidence** while generating no **parent-goal progress**. This distinction is load-bearing. Commits, passing tests, completed subtasks, model calls, and status reports prove activity. Goal progress requires evidence that the desired world state has become closer, less uncertain, less risky, or less costly to reach.

Persistent workers amplify the gap. A well-scoped prompt, once scheduled, tends to preserve its local objective. Without a means–ends review layer, the worker can faithfully execute an obsolete or strategically dominated plan for days. The scheduler sees health; the human experiences drift.

1.1 The missing feedback loop

The missing loop has four components:

1. maintain explicit top-level goals and priorities;
2. connect tasks to those goals through reusable intermediate goals;
3. update the links when results, assumptions, costs, or priorities change;
4. repeatedly test whether active tasks remain relevant and well-shaped.

This is not full general intelligence. It is a disciplined approximation to cross-task means–ends reasoning, opportunity-cost awareness, and reflective control. It can remove a consequential class of failures even when each local worker remains an ordinary LLM agent.

1.2 How to read the project-specific examples

The motivating incidents occurred in a research workspace containing persistent software agents, scheduled experiments, and several related symbolic-AI tools. The names are incidental to the argument:

- **OpenClaw** is the host agent platform that schedules and runs persistent workers.
- **OmegaClaw** is an experimental family of conversational research agents built from an upstream codebase plus preserved memories and selected capabilities.
- **PeTTa** and **SWI-Prolog** are parts of the symbolic-language runtime used by some experiments and agents.
- **Chroma** is a vector database used to store and retrieve agent memories.

Readers can mentally substitute “scheduled implementation worker,” “long-running research experiment,” “shared runtime,” “production service,” and “memory database.” The proposed governor is not specific to any of these products.

2 Motivation from observed history

The examples below are not hypothetical reconstructions. They are concrete failures that motivated the proposal. They also show why task correctness is not enough: each task had a locally intelligible rationale.

2.1 Migration fragmented into ten-minute slices

A migration intended to finish quickly was decomposed into short, repeated execution slices. Short slices offered recoverability, frequent observation, and bounded damage. But they also imposed repeated startup, rehydration, inspection, reporting, and scheduling overhead. The worker optimized a local safety mechanism without periodically comparing it against the parent goal: complete the migration promptly.

The failure was not simply choosing safety over speed. The deeper failure was not presenting the tradeoff. A relevance review should have compared at least three alternatives:

- frequent short slices: highest observability, highest overhead;
- one continuous run with checkpoints: lower overhead and recoverable;
- one uninterrupted run: fastest, with the largest recovery cost.

Given a high priority on completion speed, the system should have proposed the continuous checkpointed path without waiting for the human to notice the cadence problem.

2.2 An implementation worker pursuing an achieved milestone

A scheduled code-generation worker, internally called Plain2MeTTa, continued targeting an earlier demonstration milestone that had already been completed and immutably tagged. The worker continued to emit nominally successful runs and encountered incidental shell and test-path failures, but no current top-level goal was advanced. The actual next goal was a revised second-version intermediate representation for the translation pipeline.

This should be a nearly deterministic stop condition. If a task’s sole parent goal is marked achieved, the task is unschedulable until it is closed or linked to a new unmet goal. A simple graph query would have exposed the stale edge:

```
task: scheduled-code-generation-worker
  contributes_to -> milestone:initial-demonstration [status=achieved]
  contributes_to -> (none active)
verdict: STOP_STALE; request retargeting
```

2.3 A long-running research experiment blocking urgent service restoration

A long-horizon computational-chemistry experiment, internally called petta-chem, occupied a shared symbolic-language runtime and source tree. A clean restoration of three broken conversational agents was treated as blocked until that experiment finished, even though service restoration was urgent and the chemistry run was checkpointed and resumable. Once the conflict was made explicit, the experiment was paused after 242 of 276 durable work units, its completed results were preserved, and the agent-restoration work could proceed.

The missing information was not buried in either task. It was the cross-task relation:

```
task: checkpointed-chemistry-calibration
  occupies -> resource:shared-symbolic-runtime
  supports -> goal:long-horizon-research

task: clean-agent-service-restoration
  requires -> resource:shared-symbolic-runtime-quiescent
  supports -> goal:restore-conversational-agents [priority=urgent]

derived: chemistry-calibration blocks service-restoration
recommended verdict: PAUSE_RECOVERABLY(chemistry-calibration)
```

The correct response did not require deep chemistry or runtime intelligence. It required representing resource contention, priority, and reversibility in one decision context.

2.4 An over-engineered agent-repair path

The agent-repair effort itself accumulated a privileged process inspector, several layers of launch wrappers, synthetic safety-boundary tests, and repeated local review gates. Many components addressed real safety concerns. Yet the chosen path became dominated by machinery whose completion was not necessary for the top-level goal: restore three usable conversational agents from a clean copy of the maintained upstream software while preserving valuable memories.

A relevance review should repeatedly ask whether a simpler path now dominates: fresh upstream installation, three isolated agent identities, ordinary conversation tests, disposable copies of the memory database for compatibility testing, and selective migration of useful capabilities. This illustrates **plan-level staleness**: every subtask may remain useful in isolation while the overall decomposition becomes an inferior route.

2.5 Premature hardening of research infrastructure

Several research and agent-infrastructure lanes accumulated increasingly specific hardening tasks: path and encoding guards, lifecycle edge cases, bounded audit behavior, generalized supervisors, and ever larger regression suites. Many changes were individually sound. The alignment question was often left implicit: was the hardened component already known to support a scientifically or operationally valuable result?

For exploratory research, the next useful result is usually epistemic. It may be a calibrated estimator, a planted-signal recovery, a meaningful baseline comparison, a falsifying counterexample, or a first credible scientific signal. Generalized production hardening has weak value if the underlying method may soon be discarded. The correct early standard is not carelessness; it is **minimum sufficient rigor**: enough correctness, provenance, controls, reproducibility, cost containment, security, and recoverability to make the next result trustworthy.

This distinction challenges hardening tasks with a counterfactual:

If this hardening is omitted now, does the next scientific or operational result become invalid, uninterpretable, irreproducible, unsafe, unaffordable, or unrecoverable?

If not, the task should normally be deferred until evidence justifies a reusable research tool or production system. A large safety suite can be essential for a production delegation substrate while being irrelevant to an earlier question such as whether a proposed algorithm produces any useful signal.

3 Other failure classes the governor should catch

The same structure applies beyond the observed incidents.

Failure class	Example and expected detection
Goal completion drift	Tests keep improving a feature after its acceptance criterion passed. Detect active tasks whose parent goals are achieved.
Goal supersession	A new architecture replaces the old one, but old workers continue. Detect edges to superseded goals and plans.
Priority inversion	A low-value benchmark monopolizes a GPU, database, or human review channel required by an urgent incident. Derive a blocks relation.
Proxy gaming	A worker increases test counts or commits while the actual quality metric is unchanged. Separate activity from goal-delta evidence.
Sunk-cost continuation	Repeated failures accumulate, but prior investment is treated as a reason to continue. Recompute prospective value, not historical cost.

Failure class	Example and expected detection
Risk drift	A once-safe task becomes dangerous after scope, permissions, or external state changes. Trigger review on assumption and resource changes.
Cost drift	API usage, runtime, or human attention exceeds the initial estimate. Trigger review on material estimate variance.
Dependency drift	An upstream release obviates custom work, or a dependency becomes unavailable. Re-evaluate necessary-for and alternative-path edges.
Premature optimization	Performance work begins before correctness or estimator validation. Enforce prerequisite and necessary-for relations.
Coordination overload	Excessive checkpoints, deliberations, or reports slow the goal they protect. Model governance itself as a cost-bearing task.
Human bottleneck creation	The agent requests repeated copy/paste, approval, or archaeology that could be automated. Represent human attention as scarce.
Silent tradeoff choice	The worker selects safety, speed, quality, or cost without reporting the alternatives. Require explicit tradeoff evidence.

4 Stage-sensitive goals and result contracts

Top-level priorities are necessary but do not completely specify rational work. A goal such as “reach a mature useful result quickly” can be interpreted as permission to polish indefinitely. The governor therefore needs three general meta-goals plus an explicit definition of the result being pursued now.

4.1 Three sharpened meta-goals

A: Decision-relevant speed

Reach the next result that can rationally change belief, direction, or action as quickly as possible.

B: Least commitment

Prefer the simplest, most reversible, and least-committing path that can produce a trustworthy result.

C: Mature-result efficiency

Reach the intended mature result as quickly as feasible, avoiding shortcuts whose probability-weighted downstream cost exceeds their present value.

B is an Occam heuristic, not an absolute command. “Simplest” means simplest subject to the evidential and operational validity required at the current stage. It does not justify a shortcut that makes an experiment uninterpretable or risks destruction of valuable data. C prevents A from becoming naive short-termism, but C must not be used to fund speculative maturity before the project has earned it.

4.2 Useful results depend on project kind

The phrase *useful result* has different meanings in different projects.

Project kind	Typical next useful result	Typical mature useful result
Exploratory research	A decision-changing signal, falsification, calibrated probe, or meaningful baseline result	A replicated and well-delimited scientific conclusion; hardened software may remain a separate project
Confirmatory research	Independent replication, stronger controls, or a pre-declared test	A robust result with uncertainty, ablations, limitations, and reproducible artifacts

Project kind	Typical next useful result	Typical mature useful result
Prototype engineering	A thin end-to-end slice proving the central user or system capability	A reusable implementation adequate for repeated trials
Production engineering	A bounded service path under representative load	A secure, observable, maintainable, fault-tolerant deployed system
Service recovery	Restoration of the smallest standard known-good path	A stable restored service with preserved state and selectively reintroduced features
Infrastructure research	Evidence that the substrate enables a valuable downstream task	A reusable substrate whose hardening is proportional to its demonstrated use

An exploratory project’s mature result is often a successful or negative scientific conclusion, not hardened code. Once the scientific value is clear, building a reusable or production implementation can become a distinct project with its own goal, budget, acceptance criteria, and risks.

4.3 A maturity ladder

A useful default ladder is:

E0 – Exploration

Run the smallest trustworthy experiment that can falsify or support the core idea.

E1 – Signal validation

Calibrate the estimator, add decisive controls, and establish one credible result.

E2 – Replication and robustness

Test seeds, datasets, baselines, ablations, perturbations, and uncertainty.

E3 – Reusable research implementation

Add the cleanup, documentation, interfaces, and tests required for repeated scientific use.

E4 – Production engineering

Add security, fault tolerance, deployment, monitoring, performance, and broad edge-case hardening.

The labels are defaults, not a universal ontology. A service-recovery project may use a parallel ladder: unchanged known-good baseline, isolated instances, state compatibility, selective feature return, and production soak. What matters is that each active task names the stage it advances.

4.4 The result contract

Each active project or major goal should maintain:

1. project kind;
2. current phase or maturity stage;
3. next decision-relevant result;
4. evidence sufficient for that result to be trustworthy;
5. intended mature result;
6. known shortcuts and their likely debt;
7. conditions under which additional hardening becomes necessary;
8. explicit non-goals for the current phase;
9. owner, provenance, confidence, and next review trigger.

The contract is not frozen forever. A meaningful result, failure, cost change, or new alternative should trigger reconsideration. An LLM can propose the contract from project records, but consequential definitions and priority changes should initially be confirmed by the human owner.

4.5 Minimum-sufficient-hardening test

Hardening at E0 or E1 is presumed deferred unless omitting it materially threatens one of the following for the next result:

- scientific validity or estimator correctness;
- interpretability of the result;
- reproducibility and provenance;
- protection against silent data corruption;
- cost containment or recoverability of expensive work;
- security, privacy, or explicit authorization boundaries;
- the ability to run the discriminating experiment at all.

If the test passes, implement the smallest guard that protects the result. If it fails, record a deferred-hardening edge and the event that should reactivate it. This preserves useful technical debt as an explicit option rather than either ignoring it or paying it prematurely.

4.6 Comparing the first result with the mature result

The governor should compare a small set of alternative plans using at least:

- expected time to the next decision-relevant result;
- probability that the result is valid and informative;
- probability-weighted rework and technical debt;
- risk of losing data, violating constraints, or creating lock-in;
- opportunity cost imposed on other goals;
- governance and coordination overhead;
- option value retained by a reversible prototype.

The estimates may initially be qualitative. Their purpose is to expose the tradeoff, not manufacture false precision. A continuous checkpointed migration may dominate both tiny repeated slices and an unsafe uninterrupted run. A small amount of architecture can reduce total time to E3, but only when its expected value exceeds the chance that E0 shows the idea should be abandoned.

4.7 Applying the contract to conversational-agent recovery

For the failed service-repair effort, the result contract should have read:

```
project_kind: service_recovery
current_stage: unchanged_known_good_software_baseline
next_result:
  ordinary bounded multi-turn conversation on a pinned upstream version
trust_evidence:
  deterministic smoke, bounded lifecycle, no orphan processes
mature_result:
  three reliable isolated agents with preserved memories and selected skills
current_non_goals:
  generalized process inspector; custom deferred runner; broad supervisor
```

Here “pinned upstream” means an exact, recorded version of the maintained base software, before custom extensions are restored. Against that contract, a generalized privileged process inspector contributes mainly to a hypothetical later operational maturity stage. A clean pinned upstream installation tests the central uncertainty directly, is more reversible, uses fewer assumptions, and reaches the next result sooner. The correct verdict is **DEFER** for the inspector and **DO_FIRST** for the clean baseline.

5 The proposed control structure

5.1 A typed, evidence-bearing graph

The minimal persistent object is a directed property graph. Text can be the initial semantic payload, but node and edge types should be explicit.

Core node types.

- **Goal:** a desired world state with priority, provenance, time horizon, success criteria, status, and authority.
- **Intermediate goal:** a reusable enabling state between strategic goals and concrete work.
- **Task:** an executable commitment with owner, expected duration, cost, risk, reversibility, acceptance test, and evidence path.
- **Result:** an observed outcome, artifact, measurement, failure, or decision that updates beliefs about the graph.
- **Resource:** CPU, GPU, process tree, repository, identity, credential, database, network endpoint, or human attention.
- **Constraint:** safety, budget, authorization, deadline, ordering, privacy, semantic, or lifecycle invariant.
- **Plan:** a candidate decomposition or path through intermediate goals and tasks, allowing competing plans to be compared.
- **Result contract:** project kind, active stage, next result, trust evidence, mature result, current non-goals, and hardening trigger.

Core edge types.

- `contributes-to`, `necessary-for`, and `sufficient-for`;
- `depends-on`, `blocks`, `conflicts-with`, and `competes-for`;
- `achieves`, `invalidates`, `supersedes`, and `provides-evidence-for`;
- `alternative-to`, `refines`, and `part-of`;
- `authorized-by`, `proposed-by`, and `reviewed-by`.

Each uncertain edge should carry confidence, provenance, assumptions, last review time, and a criterion for revisiting it. A bare weight is inadequate: two equal weights can mean very different things if one rests on direct evidence and the other on an LLM guess.

5.2 Priorities are not merely scalar

A weighted score is useful for ranking, but some relationships should be lexicographic or constraint-based. For example:

1. do not violate explicit safety and authorization constraints;
2. address urgent production restoration before discretionary research;
3. among allowed tasks, maximize expected progress per scarce resource;
4. prefer simpler, more reversible plans when expected value is similar.

An illustrative task value is

$$V(t) = P(g) U(t, g) S(t) T(t) - C(t) - R(t) - O(t) - G(t),$$

where $P(g)$ is inherited goal priority, $U(t, g)$ expected contribution, $S(t)$ probability of success, and $T(t)$ urgency. $C(t)$ is direct cost, $R(t)$ risk, $O(t)$ opportunity cost imposed on other paths, and $G(t)$ governance/coordination overhead. This is a prompt for structured judgment, not initially a claim of cardinal precision.

5.3 Required review questions

Every active task should periodically answer:

1. Which active goal does success advance, through which intermediate path?
2. What new parent-goal delta occurred since the previous review?
3. Is the parent goal achieved, superseded, paused, or reprioritized?
4. Is a simpler, faster, cheaper, or safer route now available?
5. What resources are held, and which higher-priority paths are delayed?
6. Which assumptions or estimates have changed?
7. What are the principal speed–safety–quality–cost tradeoffs?
8. Which result contract and maturity stage does this task advance?
9. Does proposed hardening pass the minimum-sufficient-hardening test?
10. Should the task continue, accelerate, defer, pause, stop, replan, or escalate?

5.4 Operational verdicts

A review that only writes analysis will not change behavior. It must emit one of a small number of verdicts with reasons, evidence, expiry, and authority:

CONTINUE

Current plan remains proportionate and relevant.

ACCELERATE

Reduce cadence/governance overhead or allocate more resources within authorization.

PAUSE_RECOVERABLY

Preserve state and release a resource.

STOP_STALE

Parent goal is achieved, superseded, or absent.

REPLAN

A better path or invalidated assumption changes the decomposition.

DEFER

The task may matter at a later maturity stage but is not necessary for the next decision-relevant result.

ESCALATE

Human or multi-seat judgment is required.

BLOCKED

No safe authorized step is available; name the blocker.

6 When relevance is reviewed

Periodic review is necessary but insufficient. The system should review on events that can change the means–ends relation:

- before a worker starts or resumes;
- after a meaningful success, failure, or acceptance result;
- when a goal is created, achieved, paused, superseded, or reprioritized;
- when the next result, project kind, or maturity stage changes;
- when expected duration, cost, or risk changes materially;
- when a new resource conflict or blocking relation appears;
- after repeated activity with no recorded parent-goal delta;
- at a bounded periodic cadence as a backstop.

The cadence should itself be adaptive. A stable low-risk task with valid assumptions needs fewer reviews than an incident response with rapidly changing state. Reviews should be triggered by semantic

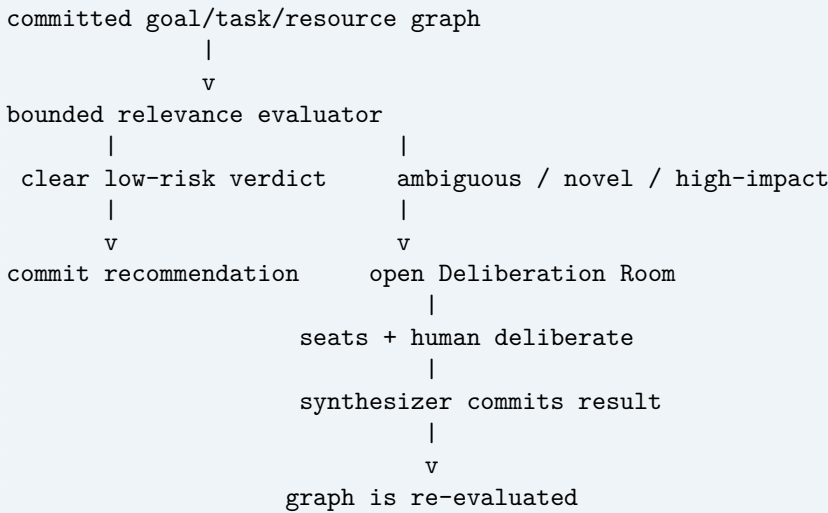
events where possible, not by arbitrary ten-minute fragmentation.

7 Relation to OmegaHive Deliberation Rooms

Deliberation Rooms and the Goal Relevance Governor solve different layers of the problem.

Dimension	Goal Relevance Governor	Deliberation Rooms
Primary question	What work remains a good means to current goals?	How do humans and seats reason together and land a conclusion?
Durable state	Goal/task/resource graph and verdict evidence	Open/close events, committed conclusion references, archived transcript
Execution shape	Cheap bounded evaluator, event-triggered or periodic	Bounded wake-based participation with chair and synthesizer
Authority	Initially advisory; later policy-gated actions	Conversation has no authority until conclusions are committed
Failure detected	Stale tasks, priority inversion, blocked resources, inferior plans	Unlanded deliberation, attribution and synthesis failures

The integration is:



Most checks should not open rooms. Making every review conversational would recreate coordination overhead. A room is appropriate when uncertainty, conflict, novelty, impact, or authority exceeds a threshold. Examples include changing a top-level priority, abandoning a major research direction, choosing between materially different safety profiles, or resolving disagreement among project seats.

The graph lives in the committed spine. A room discusses proposed changes; only the synthesizer's committed artifact changes authoritative goal state. This follows the Deliberation Rooms rule that speech is not real until it lands through the write path.

8 A simple LLM-centric OpenClaw incarnation

8.1 Storage and projections

The first version should be intentionally small:

- committed JSON or YAML records for goals, nodes, edges, and verdicts;

- stable identifiers and append-only event history;
- projections generated from existing `PROJECT.md`, `TASKS.md`, `DECISIONS.md`, cron jobs, sessions, and resource observations;
- textual descriptions on nodes, with typed metadata around them;
- source pointers for every factual status and uncertain inference.

Project Markdown remains the human-readable source of project facts. The graph can initially be a normalized cross-project projection rather than a second competing truth store. Explicit goal and priority records are added only where the current project files lack them.

8.2 LLM roles

Separate LLM calls can perform bounded functions:

1. **Link proposer**: suggests intermediate goals and typed edges.
2. **Result-contract proposer**: classifies project kind and stage, proposes next and mature results, sufficient evidence, and non-goals.
3. **Evidence checker**: verifies whether claims have source support.
4. **Relevance critic**: answers the required review questions.
5. **Alternative planner**: proposes simpler or faster paths.
6. **Adversarial reviewer**: searches for priority inversion, opportunity cost, hidden blockers, and silent tradeoffs.
7. **Synthesizer**: emits a structured recommendation or prepares a Deliberation Room packet.

These may initially be prompts to one strong model, but their contracts should remain distinct. The evaluator must not silently edit human priorities. It may propose changes with confidence and provenance.

8.3 Authority ladder

- **Read-only shadow**: record what would have been recommended.
- **Advisory**: notify the operator of stale or conflicting work.
- **Scheduling gate**: prevent a worker with no active goal path from starting, while allowing explicit override.
- **Low-risk autonomy**: pause checkpointable work or retarget an obviously stale worker under pre-declared rules.
- **Strategic autonomy**: defer until extensive validation; major goal and priority changes remain human-authoritative.

This progression prevents a relevance governor from becoming another source of global damage. It should earn authority through counterfactual evidence.

9 A symbolic reasoning incarnation

9.1 Atomspace representation

An Atomspace is a typed knowledge graph whose nodes and links can themselves participate in reasoning. The same interface can be represented as atoms rather than JSON objects. A schematic MeTTa-like symbolic vocabulary might include:

```
(Goal RestoreConversationalAgents)
(Priority RestoreConversationalAgents urgent)
(Status RestoreConversationalAgents active)

(Task ChemistryCalibration)
```

```

(Task CleanAgentInstallation)
(ContributesTo ChemistryCalibration LongHorizonResearch)
(ContributesTo CleanAgentInstallation RestoreConversationalAgents)
(Occupies ChemistryCalibration SharedSymbolicRuntime)
(Requires CleanAgentInstallation QuiescentSharedSymbolicRuntime)
(Checkpointable ChemistryCalibration True)

(ProjectKind AgentRecovery ServiceRecovery)
(CurrentStage AgentRecovery KnownGoodBaseline)
(NextResult AgentRecovery OrdinaryBoundedConversation)
(MatureResult AgentRecovery ThreeIsolatedAgentsWithPreservedMemory)
(CurrentNonGoal AgentRecovery GeneralizedProcessInspector)

(Evidence E1 (CompletedUnits ChemistryCalibration 242 276))
(Confidence (Blocks ChemistryCalibration CleanAgentInstallation) 0.99)

```

Intermediate goals become first-class atoms. Rules can derive transitive contribution, blocked paths, alternative paths, and goal achievement. Temporal atoms can represent deadlines, expiry, last review, and changing status.

9.2 PLN-style uncertain inference

Probabilistic Logic Networks (PLN), a framework for uncertain logical inference over such graphs, can add more than a symbolic transcription. It can combine uncertain means–ends links and evidence:

- infer the strength with which a task contributes to a goal through several uncertain intermediate links;
- weaken a contribution when repeated results fail to change the parent state;
- compare competing plans using uncertain success and cost estimates;
- reason separately about time-to-information and time-to-mature-result;
- propagate new evidence that invalidates assumptions;
- distinguish lack of evidence from evidence of irrelevance;
- explain derived verdicts as inference trails.

A simple conceptual rule is:

If task T contributes to intermediate goal I , and I contributes to goal G , then T contributes to G with an uncertainty derived from both links; new evidence about either link updates the composite relevance.

The numerical semantics require care. Contribution is not ordinary truth, and priority is not probability. It may be better to use several typed values: belief in the causal link, expected utility, urgency, cost, and confidence in the estimate. PLN should reason over these without collapsing them into one opaque scalar too early.

9.3 Existing components and their roles

The governor should reuse rather than replace existing components:

- **ThreadKeeper**, a persistent-worker lifecycle service, supplies durable task events and worker-status primitives.
- **GoalChainer**, an experimental symbolic goal-appraisal module, supplies norm/goal/task relations and uncertain belief grading seams.
- **Petta-memory**, a symbolic memory layer, can preserve evidence and attribution needed by uncertain relevance inference.

- **Deliberation Rooms**, a bounded multi-agent discussion format, supply strategic review when routine evaluation is insufficient.
- **Conversation Governor**, a message-control layer, handles message admission and egress; it is adjacent but does not decide which tasks are strategically useful.

The abstraction seam is the typed graph plus verdict contract. Storage, inference, and transport can change without rewriting scheduling policy.

10 Validation before authority

10.1 Retrospective replay corpus

The initial detector should be evaluated against frozen historical episodes, including at least:

1. migration slice overhead before the continuous-run proposal;
2. the stale code-generation worker pursuing an achieved milestone;
3. a checkpointed chemistry experiment blocking urgent service recovery;
4. the over-engineered conversational-agent repair path;
5. premature hardening tasks before evidence of research value;
6. matched controls where long-running or cautious work was justified.

For each episode, freeze only information available at the review time. A successful system must flag the problem before the human intervention, name the correct parent-goal issue, and propose a proportionate verdict. It must not use later outcomes as hidden evidence.

10.2 Metrics

- detection recall for known alignment failures;
- false-alarm rate on justified long-running tasks;
- decision latency from decisive evidence to recommendation;
- estimated time and resource savings under counterfactual replay;
- human agreement with the stated tradeoff and verdict;
- correct classification of project kind, stage, and useful result;
- hardening precision: necessary guards retained, premature polish deferred;
- explanation completeness and evidence traceability;
- governor overhead relative to savings;
- rate of reviews producing no new information.

The last two protect against a recursive failure: building a large governance system whose cost exceeds the drift it prevents.

10.3 Shadow-first deployment

Deployment should proceed from offline replay to read-only shadow observation, then advisory recommendations, and only later to narrow scheduling gates. No live autonomy is justified merely because the architecture is persuasive. Predeclared promotion thresholds and explicit rollback should govern each authority increase.

11 Design risks and safeguards

11.1 Bureaucratic overgrowth

The graph can become a ceremonial ontology maintained more carefully than the work itself. Start with the few node/edge types needed to detect actual failures. Every field should support a query or decision.

Measure governance overhead.

The same warning applies to maturity models. The E0–E4 ladder is a diagnostic aid, not a reason to create five phases of ceremony for every small task.

11.2 LLM confabulation and unstable links

LLMs may invent causal relationships or rewrite priorities in rhetorically plausible ways. Separate proposals from accepted links; attach evidence, confidence, author, and expiry; use adversarial review for major edges; and keep human priorities authoritative at first.

11.3 Priority laundering

Derived scores can conceal value judgments. Preserve the original human goal and priority statement, show every inherited priority path, and expose the tradeoff dimensions before aggregation.

11.4 Over-centralization

A global governor can become a brittle bottleneck. Projects should retain local autonomy within explicit resource and priority bounds. The global layer intervenes mainly on cross-project conflicts, stale parent goals, and major plan changes.

11.5 Premature enforcement

Incorrect pauses may destroy momentum or invalidate experiments. Begin read-only; prefer recoverable pauses; require explicit authority for irreversible actions; and retain human override with a recorded rationale.

12 A staged implementation path

Phase 0: Design and frozen examples

Complete this specification; define a small retrospective corpus and expected verdicts; freeze result contracts for the motivating cases; identify existing data sources and invariants.

Phase 1: Read-only graph projection

Project current goals, tasks, cron workers, sessions, and resource conflicts into a typed JSON graph. Include project kind, maturity stage, next result, and current non-goals. Do not alter scheduling.

Phase 2: LLM relevance critic

Run event-triggered and periodic reviews. Emit structured shadow verdicts with evidence and alternatives. Evaluate replay metrics.

Phase 3: Deliberation integration

Open rooms for ambiguous or high-impact cases. Require committed synthesis and measure landing rate and decision latency.

Phase 4: Narrow scheduling gates

With explicit approval, block only deterministic stale cases, such as a task whose sole parent goal is achieved. Preserve overrides and rollback.

Phase 5: Atomspace/PLN realization

Map the stable graph and verdict interface into atoms in a symbolic knowledge graph. Compare symbolic and LLM judgments on the same corpus. Use PLN for uncertain contribution and evidence propagation where it demonstrably improves results.

13 Open research questions

1. What is the smallest graph vocabulary that catches the motivating failures without becoming a general project-management ontology?
2. Can an LLM reliably infer a project’s kind, current stage, and next decision-relevant result from incomplete project records?
3. How should contribution, belief, utility, priority, urgency, and risk be represented without inappropriate scalar collapse?
4. How can intermediate goals be reused without erasing project-specific semantics?
5. What event thresholds trigger review without reproducing excessive checkpoint overhead?
6. How should a relevance evaluator estimate opportunity cost when alternative plans are only partially known?
7. How should technical debt and option value be estimated without systematically favoring either premature hardening or reckless prototypes?
8. Which verdicts can safely execute automatically, and which should always open a Deliberation Room or request human approval?
9. Can embedding-based conceptual similarity propose useful reusable intermediate goals while provenance and logic prevent false analogy?
10. Does PLN improve calibration and explanation over a strong LLM critic, or mainly provide a more inspectable representation?

14 Conclusion

The motivating failures share a single structure: local tasks continued under fixed assumptions while the global meaning of the work changed. A Goal Relevance Governor addresses this by making means–ends links, priorities, resources, evidence, and alternatives explicit and repeatedly reviewable. Its essential output is not another report but an operational verdict tied to current goals.

The result contract adds a necessary temporal and epistemic dimension. It distinguishes the result needed now from a hypothetical mature artifact, and it distinguishes validity-critical rigor from premature hardening. The governor can therefore optimize for fast trustworthy learning without losing sight of downstream maturity.

The practical first version does not require a mature symbolic AGI. Committed textual records, a small typed graph, bounded LLM critics, and retrospective validation can already catch stale goals, priority inversions, blocked resources, and inferior decompositions. The same architecture can later move into Atomspace and PLN without changing its outer contract.

Deliberation Rooms strengthen rather than replace the proposal. They provide a bounded venue where humans and seats exercise consequential judgment. The graph and review workers decide when that judgment is needed and preserve its committed consequences. Together they form a plausible incremental path from task-local LLM competence toward persistent reflective intelligence across projects.

Source and provenance notes

This document synthesizes Ben Goertzel’s 2026-08-14 proposal and the immediate discussion with ZeroBot. The Deliberation Rooms comparison is based on Cassio Pennachin, *OmegaHive Deliberation Rooms: Vision, a Slack MVP, and the relation to the Conversation Governor*, July 29, 2026, preserved locally with SHA-256 8c6d6b3916d7353b5e7818fab2a20abe98d8c8a6d553df936316fec44419e762. His-

torical examples are operational observations from the shared OpenClaw workspace; they are used here as design evidence, not as claims that the proposed mechanism has already been validated.